PilotFish HL7 Validation Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **spell-check for HL7 messages**. The processor reads the pipe-delimited HL7 payload, asks the HAPI library whether the structure parses cleanly, and records pass/fail on the transaction. The original message bytes are preserved for downstream steps.

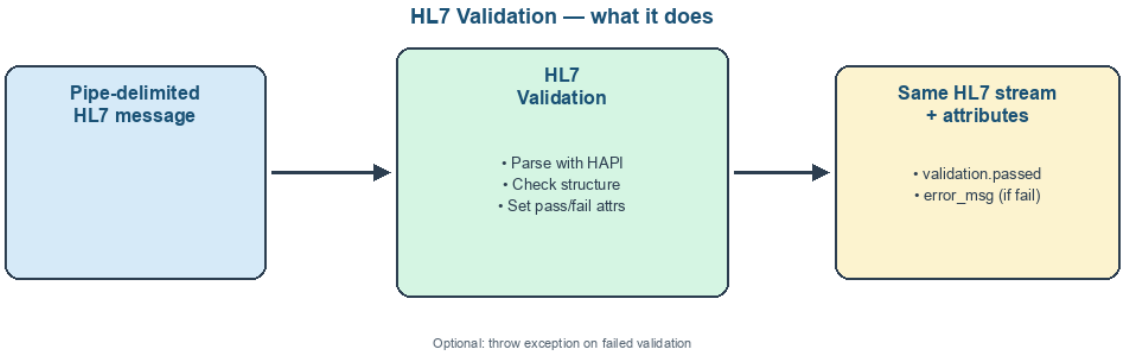

Figure 1: At a glance (plain English)

- **Parses** the transaction stream with HAPI's pipe parser.
- **Records** whether validation passed in a boolean attribute.
- **Captures** the parser error message when validation fails.
- **Optionally throws** so the route can fail fast instead of continuing.

Purpose

This document is a **deep dive** into the **HL7 Validation** processor as shipped in **`eip.war.hs.26R1.11`**: how it buffers HL7 text, validates with HAPI, sets result attributes, and optionally raises an exception.

Provenance	Value
WAR	`eip.war.hs.26R1.11`
Module JAR	`xcs-format-hl7-1.0.1.jar`
Decompiler	CFR 0.152
Implementation class	`com.pilotfish.eip.modules.hl7.HL7Validatio`
Decompiled path	`war-pipeline/decompiled/eip-26R1.11/.../HL`
Processor type string	`HL7 Validation`
Description	`Validates a pipe-delimited HL7 message`
Help ID	`console.processor.hl7_validation_processor`
Categories	`HL7`, `VALIDATION`, `HEALTHCARE`

Derived from WAR bytecode (CFR). Narrative follows executable logic in `processData`.

---

1. Conceptual model

HL7 Validation is a **non-destructive check** on raw HL7 text:

```
HL7 stream
     |
     v
 EIP default cache  (full copy)
     |
     +-- InputStreamReader (configured encoding) --> HAPI PipeParser.parse(...)
     |                                          |
     |                                          v
     |                          attributes: validation.passed, error_msg (on fail)
     |
     +-- cache.getInputStream() --> data.setDataStream(...)  (restored)
```

It does **not** transform HL7 to XML, send ACKs, or modify segment content.

---

2. High-level behavior (`processData`)

1. **Cache the payload**
   - `EIPCacheManager.getInstance().getDefaultCache(data)`
   - `DataUtil.copyAndClose(data.getDataStream(), cache.getOutputStream())`
   - `data.setDataStream(cache.getInputStream())`

    - Copy failure -> `EIPException("Error copying transaction contents to cache")`
2. **Open a reader** on the cache stream using **Character Encoding** (`CharacterEncoding` tag).
3. **Parse with HAPI**
    - `DefaultHapiContext` / `context.getPipeParser().parse(IOUtils.toString(reader))`
    - Success -> set `com.pilotfish.hl7.message.validation.passed` = `true`
4. **On parse failure** (`Exception` catch-all):
    - Set `validation.passed` = `false`
    - Set `com.pilotfish.hl7.message.validation.error_msg` = `e.getMessage()`
    - If **Throw Exception On Failed Validation** is true -> `EIPException("HL7 message validation failed: " + message)`
5. **Return** the same `TransactionData` with stream restored from cache.

`systemShutdown()` closes the HAPI context; I/O errors during close are printed to stderr.

---

3. Configuration reference

3.1 Character Encoding

Property	Value
UI label	Character Encoding
Tag	`CharacterEncoding`
Type	String
Default	`UTF-8`
Required	yes
Tooltip	Encoding of HL7 message content being validated

Nuances:

- Passed to `InputStreamReader` when reading cached bytes.
- Wrong encoding can cause false validation failures or mis-read characters before HAPI sees the text.

3.2 Throw Exception On Failed Validation

Property	Value
UI label	Throw Exception On Failed Validation
Tag	`ThrowException`
Type	Boolean
Default	`false`
Required	no
Tooltip	Whether or not to throw an exception in addition to setting validation result transaction attributes

Nuances:

- When `false`, failed validation is **soft**: attributes record the failure and the route continues.
- When `true`, attributes are still set before the exception is thrown.

3.3 Inherited AbstractProcessor options

Property	Value
UI label	Execute Processor
Tag	`ExecuteProcessor`
Type	Boolean
Default	`true`
Tab	Conditional Execution

When skipped, no caching, parsing, or attributes are produced.

---

4. Transaction attributes

Attribute key	Type	When set
`com.pilotfish.hl7.message.validation.passed`	Boolean	Always on successful `processData` completion
`com.pilotfish.hl7.message.validation.error_msg`	String	Only when validation fails

Both are registered in `getConfigurationDescriptor()` for eiConsole discovery.

---

5. Worked examples

5.1 Soft validation branch

- **Throw Exception** = `false`
- Valid HL7 -> `validation.passed` = `true`; route continues.
- Invalid HL7 -> `validation.passed` = `false`, `error_msg` populated; route can branch on the boolean.

5.2 Hard stop on bad messages

- **Throw Exception** = `true`
- Invalid HL7 -> route error handling receives `EIPException` with HAPI message text.

---

6. Known quirks and caveats

1. **Pipe-delimited only** — uses HAPI `PipeParser`, not XML or ER7 variants unless HAPI accepts them.
2. **Structural validation** — confirms HAPI can parse; does not replace full profile/constraint checking.
3. **Any `Exception` counts as failure** — broad catch; inspect `error_msg` for root cause.
4. **Stream consumed into cache** — original stream object is closed/replaced; downstream steps use the restored cache stream.
5. **HAPI context lifecycle** — one context per processor instance; closed on system shutdown.

---

7. Operational recommendations

Goal	Guidance
ACK routes	Use soft validation + branch; reserve hard throw for unrecoverable input
Encoding	Match listener/source encoding (often UTF-8 or site-specific)
Downstream XML	Run HL7-to-XML **after** validation if both are needed
Monitoring	Log or route on `validation.passed` and `error_msg`
Conditional skip	Use **Execute Processor** when only some message types need validation

---

8. Source index (this WAR)

Topic	Path under `war-pipeline/decompiled/eip-26R1.11/`
HL7 Validation processor	`com/pilotfish/eip/modules/hl7/HL7ValidationProces`
AbstractProcessor	`com/pilotfish/eip/extend/AbstractProcessor.java`
Module JAR	`war-pipeline/jars/eip-26R1.11/xcs-format-hl7-1.0.`

---

9. Pipeline provenance

```
PilotFish WARs/eip.war.hs.26R1.11
  -> xcs-format-hl7-1.0.1.jar
  -> CFR decompile
  -> Documents/Processors/26R1.11/PilotFish-HL7-Validation-Reference-26R1.11.(md|pdf)
```

Deep-dive documentation from WAR decompile – HL7 Validation Processor – 26R1.11 – August 2026.